

In this chapter, when we talk about how to install a replication system, we will not use Docker containers; this is because to install a replication service, we need to shut down the postgresql service on the replica server, and shutting down a service in a Docker environment shuts down the whole container. However, on Docker, shutting down postgresql and restarting it at the same time can be done when the Docker container starts for the first time, so to better understand how to install a replica on a production server, it is better not to use a Docker container (even if on the GitLab repo you can find some containers that you can use). In this chapter, Docker is only used to help the reader who doesn't want to install a replica server to learn and instead wants to see how physical replication works; for everything concerning the installation part, we will refer not to the paths of the Docker images but to the paths of a Debian server.

Technical requirements

In the *Learn PostgreSQL* GitHub repository, you can find three Docker images:

- **chapter17_streaming**: Primary/replica asynchronous replication; if you want to use the replica container after starting the container with:

```
chapter_17$ bash run-pg-docker.sh chapter17_streaming
```

you have to run:

```
chapter_17$ bash run-pg-docker_replica.sh chapter17_streaming
```

- **chapter17_synchronous**: Primary/replica synchronous replication; if you want to use the replica container after starting the container with:

```
chapter_17$ bash run-pg-docker.sh chapter17_synchronous
```

you have to run:

```
chapter_17$ bash run-pg-docker_replica.sh chapter17_synchronous
```

- **chapter17_delayed**: Primary/replica with delayed replication; if you want to use the replica container after starting the container with:

```
chapter_17$ bash run-pg-docker.sh chapter17_delayed
```

you have to run:

```
chapter_17$ bash run-pg-docker_replica.sh chapter17_delayed
```

All the replica containers are stopped when we exit from the primary container.

In this chapter, we will cover the following topics:

- Exploring basic replication concepts
- Managing streaming replication

Exploring basic replication concepts

In PostgreSQL, there are two kinds of physical replication techniques:

- **Asynchronous replication:** In asynchronous replication, the primary device (source) sends a continuous flow of data to the secondary one (target), without receiving any return code from the target. This type of copying has the advantage of speed, but it brings with it greater risks of data loss because the received data is not acknowledged.
- **Synchronous replication:** In synchronous replication, a source sends the data to a target, that is, the second server; at this point, the server acknowledges that the changes are correctly written. If the check is successful, the transfer is completed.

Both methods have advantages and disadvantages, and in the *Managing streaming replication* section of this chapter, we will analyze them.

Physical replication and WALs

Let's briefly summarize what we have already covered about MVCC and WAL segments: we have seen how PostgreSQL stores data on disk using WAL segments, and as we saw in *Chapter 11, Transactions, MVCC, WAL, and Checkpoints*, WAL segments are mainly used in the event of a crash. After a crash, PostgreSQL retraces WAL segments and reapplies them to data starting from the last checkpoint; during the recovery time after a crash, the server puts itself in a recovery state. Here is a summary of the key information about WAL segments:

- The WAL size is fixed at 16 MB.
- By default, WAL files are deleted as soon as they are older than the latest checkpoint.
- We can maintain extra WAL segments using `wal_keep_segments`.
- WAL segments are stored in the `pg_wal` directory as shown here:

```
postgres@pg2:~/16/main/pg_wal$ ls -alh
total 17M
```

```
drwx----- 3 postgres postgres 4.0K May 22 09:52 .
drwx----- 19 postgres postgres 4.0K May 22 10:18 ..
-rw----- 1 postgres postgres 16M May 22 10:18
000000010000000000000001
drwx----- 2 postgres postgres 4.0K May 22 09:52 archive_status
```

The wal_level directive

The `wal_level` directive sets what kind of information should be stored in WAL segments. The default value is `minimal`. With this value, all information that is stored in a WAL segment can support archiving and physical replication.



For further information, see <https://www.postgresql.org/docs/current/runtime-config-wal.html#GUC-WAL-LEVEL>.

So, in this chapter, we will use the `wal_level=replica` value, which is the default value, and in the next chapter, we will use `wal_level=logical`. We have to remember that we need to restart the PostgreSQL server every time we change the `wal_level` parameter.

Preparing the environment setup for streaming replication

In this section, we will prepare the two servers that we need to proceed: the first one is the primary server machine, and the second one is the replica server. So, let's proceed with the installation of two virtual machines. In the following examples, we will use two Debian Linux virtual machines, with `192.168.122.10` as the IP for the primary server and `192.168.122.11` as the IP for the replication server. In this chapter, all the paths refer to a PostgreSQL 16 instance installed on Debian, for example, `/var/lib/postgresql/16/main`.

1. For the primary server, we will see the following output:

```
root@pg1# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
```

```

2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
state UP group default qlen 1000
    link/ether 52:54:00:5c:df:f4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.10/24 brd 192.168.122.255 scope global enp1s0
        valid_lft forever preferred_lft forever
    inet6 fe80::5054:ff:fe5c:dff4/64 scope link
        valid_lft forever preferred_lft forever

root@pg1:# su - postgres
postgres@pg1:~$ psql
psql (16)
Type "help" for help.

postgres=#

```

2. Similarly, for the replica server, we will have the following:

```

root@pg2:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
state UP group default qlen 1000
    link/ether 52:54:00:93:47:18 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.11/24 brd 192.168.122.255 scope global enp1s0
        valid_lft forever preferred_lft forever
    inet6 fe80::5054:ff:fe93:4718/64 scope link
        valid_lft forever preferred_lft forever

root@pg2:~# su - postgres
postgres@pg2:~$ psql
psql (16)
Type "help" for help.

postgres=#

```

3. Let's check to see whether there is a connection between the two servers.

Using the ping command, we will do a simple test to check if the node pg1 can connect to the node pg2 and if the node pg2 can connect to the node pg1:

```
postgres@pg1:~$ ping 192.168.122.11
PING 192.168.122.11 (192.168.122.11) 56(84) bytes of data.
64 bytes from 192.168.122.11: icmp_seq=1 ttl=64 time=0.292 ms
64 bytes from 192.168.122.11: icmp_seq=2 ttl=64 time=0.406 ms

postgres@pg2:~$ ping 192.168.122.10
PING 192.168.122.10 (192.168.122.10) 56(84) bytes of data.
64 bytes from 192.168.122.10: icmp_seq=1 ttl=64 time=0.536 ms
64 bytes from 192.168.122.10: icmp_seq=2 ttl=64 time=0.359 ms
```

Now that everything is ready, let's start exploring the details of physical replication.

Managing streaming replication

In this section, we will talk about why we have to have replicas.

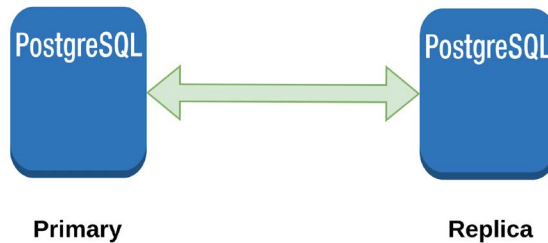


Figure 17.1: Primary/Replica Schema

In a production environment, you often need to be able to restore it as quickly as possible after a system crash. In order to do this, we have to use the streaming replication technique. To make this possible, we need at least two servers, one primary server and one secondary server. The primary server performs all the operations that will be requested by the application programs; the replica server will be available only for read operations and will have the data copied in real time.

Basic concepts of streaming replication

The idea behind streaming replication is to copy the WAL files from the primary server to another (replica) server.

The replica server will be in a state of continuous recovery, and it continuously executes the WAL that is passed by the primary machine; this way, the replica machine binarily replicates the data of the primary machine through the WAL.

As we've seen in *Chapter 15, Backup and Restore*, in a classic PITR situation, WAL segments are saved somewhere by the primary, and then they are taken by the recovery machine using manual scripts:

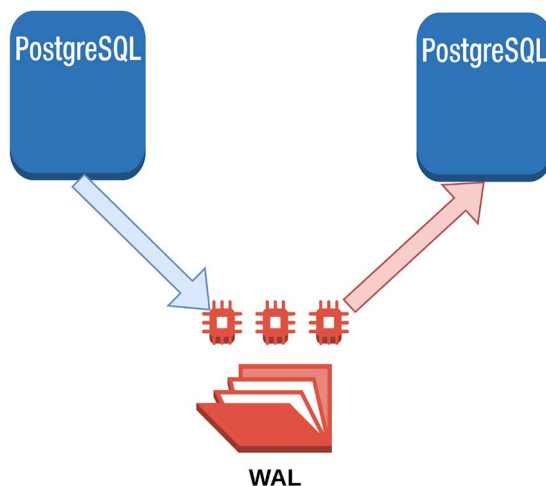


Figure 17.2: PITR Schema

In a streaming replication context, a communication channel will be open between the replica and primary, and the primary will send the WAL segments through it:

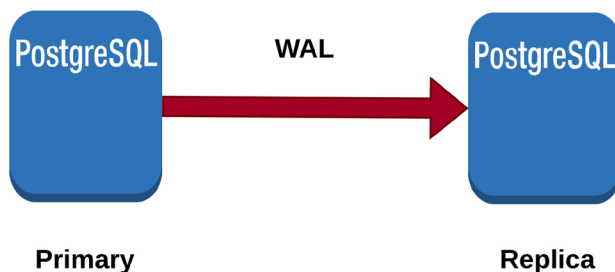


Figure 17.3: Primary/Replica WAL Schema

The replica server will receive the WAL segments and rerun them, remaining in a permanent recovery state.

We will now look at how to perform asynchronous physical replication. The technique is very similar to PITR.

Asynchronous replication environment

Let's prepare our environment. We need two servers: the first one will be called pg1, and its IP will be 192.168.122.10; the second one will be called pg2, and its IP will be 192.168.122.11. Let's take a look at the preparatory steps for physical replication.

On the primary server, we need to do the following:

1. The first thing we have to do is modify `listen_addresses` so that it listens to the network. If we set `listen_addresses = '*'`, PostgreSQL will listen to any IP; otherwise, we can specify a list of IP addresses separated by commas. This change requires a restart of the PostgreSQL service.
2. We need to create a new user that is able to perform the replication:

```
postgres=# CREATE role replicarole WITH REPLICATION ENCRYPTED
PASSWORD 'SuperSecret' LOGIN;
CREATE ROLE
```

3. We have to modify the `pg_hba.conf` file so that from the replica machine with the user `replicarole`, it is possible to reach the primary machine:

```
host      replication      replicarole      192.168.122.11/32
scram-sha-256
```

4. To make this configuration active, we need to run a reload of the PostgreSQL server. For example, we can run the following:

```
postgres=# select pg_reload_conf();
pg_reload_conf
-----
t
(1 row)
```

5. On the replica server, we have to turn off the PostgreSQL service, destroy the PGDATA directory, and remake it – this time, empty and with the right permissions. To do this, we can use these statements:

```
root@pg1:/# systemctl stop postgresql
root@pg1:/# cd /var/lib/postgresql/16/
root@pg1:/# rm -rf main
root@pg1:/# mkdir main
```

```
root@pg1:/# chown postgres:postgres main
root@pg1:/# chmod 0700 main
```

All the paths used in this example are valid for Debian-based distributions; for other distributions, please consult the respective official documentation.

The `wal_keep_segments` option

From what we have understood, physical replication is done through the transfer of WAL segments. Now suppose for a moment that the replica server goes down for some reason. How does the primary behave? When the replica server becomes functional again, will it realign itself with the primary node or not? These are questions we need to ask ourselves if we want our replication system to work correctly.

The `postgresql.conf` directive that tells PostgreSQL how many WAL segments to keep on disk is called `wal_keep_segments`; by default, `wal_keep_segments` is set to zero because the replica is not installed by the PostgreSQL installation process. This means that PostgreSQL will not store any extra WAL segments as buffers. This means that if the replica machine (standby) goes down, then it will no longer be able to realign itself when it comes back up. This happens because in the time it takes the replica to get back up, it is possible that the primary machine has produced and deleted new WAL segments. The first way to overcome this problem is to set the `wal_keep_segments` directive to a value greater than zero in `postgresql.conf`. For example, if we set a value of `wal_keep_segments = 100`, this means that at least 100 files of WAL segments will be present in the `pg_wal` folder, for a total occupied disk space of $100 * 16 \text{ MB} = 1.6 \text{ GB}$.

In this case, the primary always keeps these extra WAL segments, and if the replica should go down, then it will only be able to realign itself, once back up, if the primary has produced a number of WAL segments less than `wal_keep_segments`.

This solution offers a static buffer in that you can store old WAL segments and offers a save anchor that is shorter than the time taken by the primary to produce a number of WAL segments greater than `wal_keep_segments`. This solution is a static solution; it also has the disadvantage that the space occupied on disk is always equal to `wal_keep_segments * 16 MB`, even when it is no longer necessary to keep WAL segments on the primary server (because they have already been processed by the replica server). The advantage of this solution is that if the network goes down, PostgreSQL uses a maximum disk space equal to `wal_keep_segments * 16 MB` to avoid filling all the disk space if the primary server goes down; so if we don't have much disk space, we can use this solution, keeping in mind that if we exceed the size of `wal_keep_segments * 16 MB`, the replica will no longer be synchronized, and we will have to rebuild it.

The slot way

In PostgreSQL, there is another approach that can be used to solve the problem of storing WAL segments: the slot technique. Through the slot technique, we can tell PostgreSQL to keep all the WAL segments on the primary until they have been transferred to the replica servers. In this way, we have dynamic, variable, and fully automated management of the number of WAL segments that the primary server must keep as a buffer. This is a very easy way to manage our physical replicas, and it is the method we will focus on in this book.

The instruction we need to perform on PostgreSQL to create a new slot is as follows:

```
postgres=# SELECT * FROM pg_create_physical_replication_slot('master');
 slot_name | lsn
-----+-----
 master   |
(1 row)
```

The instruction we need to perform on PostgreSQL to drop a slot is this:

```
postgres=# select pg_drop_replication_slot('master');
 pg_drop_replication_slot
-----
(1 row)
```

Later on in this chapter, we will look at these instructions in more detail.

The pg_basebackup command

In *Chapter 15, Backup and Restore*, in the section *Basic concepts behind Point In Time Recovery*, we talked about the base backup; this is a hot backup that acts as a starting base on which we can then perform all the WAL segments. There is a command called `pg_basebackup` that implements this procedure almost automatically.

It is necessary that the `max_wal_senders` value is at least 2. It is a very useful command for the DBA because it allows us to do everything we need to do with a single instruction. We will use and better explain this command in the next section, where we will implement our first asynchronous physical replication.

For further information about the `pg_basebackup` command, please refer to <https://www.PostgreSQL.org/docs/current/app-pgbasebackup.html>.

Asynchronous replication

We now have all the building blocks necessary to easily and quickly make our first asynchronous physical replication. By default, in PostgreSQL, physical replication is asynchronous. Let's now start with the replication technique. By following the steps from the previous sections of this chapter, we already have a primary server ready to be connected to the replica server, and we have the replica ready to receive information from the primary. The replica server will now have the PostgreSQL service turned off and the PGDATA data folder created, empty, and with the right permissions granted:

1. Let's go inside the PGDATA directory as the system postgres user:

```
root@pg2:~# su - postgres
postgres@pg2:~$ cd /var/lib/PostgreSQL/16/main
```

2. Now let's run the `pg_basebackup` command with the right options. This command will execute the `base_backup` command from the primary machine to the replica machine and prepare the replica machine to receive and execute the received WAL segments, causing the replica server to remain in a state of permanent recovery:

```
postgres@pg2:~/16/main$ pg_basebackup -h 192.168.122.10 -U
replicarole -p5432 -D /var/lib/PostgreSQL/16/main -Fp -Xs -P -R -S
master
Password:
22483/22483 kB (100%), 1/1 tablespace
```

The password that we have to insert is the password of the `replicarole` user; in our case, this is `SuperSecret`. If the `pg_basebackup` doesn't start quickly, that means that it is waiting for a checkpoint from the primary, so to improve the performance of this operation, we can go on the primary server and execute:

```
postgres=# checkpoint ;
CHECKPOINT
```

Let's analyze the `pg_basebackup` command in more detail:

- `-h`: With this option, we see the host that we want the replica to connect to.
- `-U`: This is the user created on the primary server used for replication.
- `-p`: This is the port where the primary server listens.
- `-D`: This is the PGDATA value on the replica server.